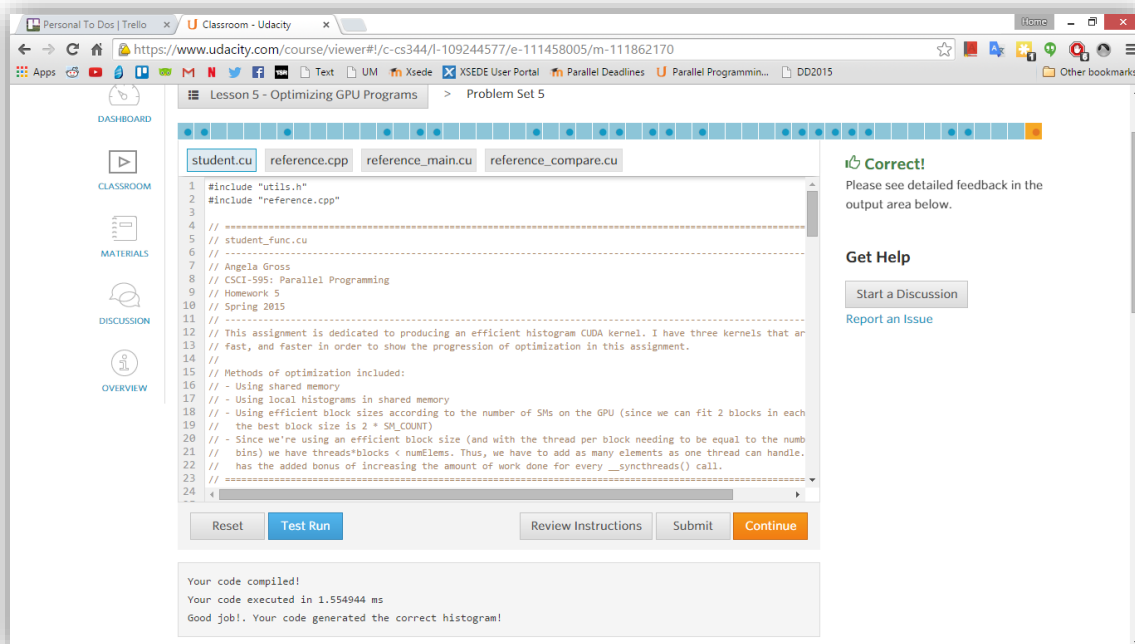
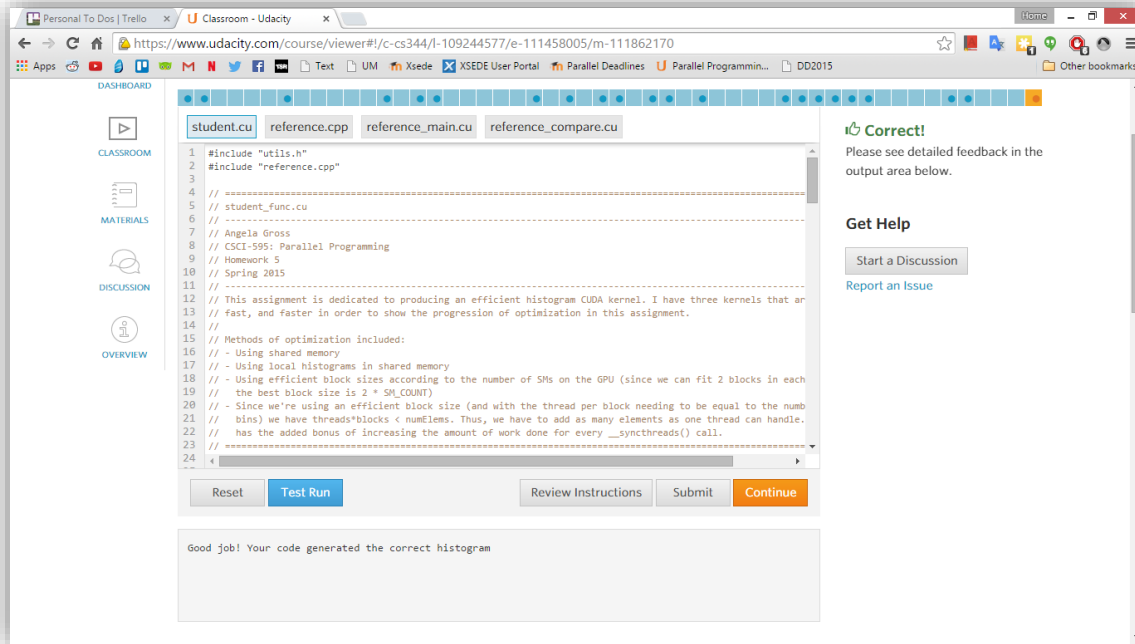


HISTOGRAM OPTIMIZATION

ANGELA GROSS // CSCI-595 // UDACITY HW 5

OVERVIEW

Homework 5 of the Udacity course asked us to implement an efficient histogram CUDA kernel, and the following two screenshots illustrate the submission (without the time) and the test run (with the time). As you can see below, the time for my fastest histogram kernel took about 1.55 ms (but averages 1.6 ms)



APPROACHES

I optimized this code in three steps using three different kernels. I first created the naïve kernel that looked something like this:

```
// retrieve the thread's global id and id and check it isn't outside the range
const int gid = blockIdx.x * blockDim.x + threadIdx.x;
if(gid >= n) return;

// atomic add by 1 into histogram
atomicAdd(&(g_histo[g_vals[gid]]), 1);
```

It simply makes use of atomic adding and adds a value to the global histogram for each thread. Thus, this kernel was launched with 256 threads and $n / 256$ blocks where n is the number of elements in the values array. 256 threads are launched at this point just because it yielded the best performance. The runtime of this kernel averaged at about 8 ms on the Udacity AWS server.

Identifying that the threads are fighting needlessly over who can access the `g_histo` array, we try to come up with an approach that will limit the number of times a thread will have to wait to access an array. We can do this by declaring a local histogram in shared memory for each block and have each thread atomically add to the local histogram. Then each thread (using their thread ID) will atomically add the local histogram bin to the global histogram bin. This will result in a better performance because (a) shared memory is faster to access and write to, and (b) the threads only have to share access to the histogram with the other threads within their block instead of with all threads in all blocks, and (c) although they will have to atomically add with the global histogram at the end, they add much less than the number of elements and so it isn't as bad.

The code looked something like this:

```
// retrieve the thread's global id and id and check it isn't outside the range
const int gid = blockIdx.x * blockDim.x + threadIdx.x;
if(gid >= n) return;

// declare external shared memory
extern __shared__ unsigned int s_histo[];

// zero out histogram and sync
s_histo[threadIdx.x] = 0;
__syncthreads();

// atomically add value in local histogram and sync
atomicAdd(&(s_histo[g_vals[gid]]), 1);
__syncthreads();

// atomically add local histogram into global histogram
atomicAdd(&(g_histo[threadIdx.x]), s_histo[threadIdx.x]);
```

When launched with 1024 threads (one thread for each bin) and $n/1024$ blocks, the runtime averaged at about 2 ms. Thus, this optimization alone resulted in a 4x speedup.

Although the last optimization gives us a fairly decent speedup, we are still being slowed down by the `__syncthreads()` call after the atomic add. One way to alleviate this is to have each thread atomically add multiple elements before syncing threads because the thread would do more work before syncing. In order to do this, however, the number of threads launched per block multiplied by the number of blocks have to be less than n (instead of greater than or equal to). We still need to have 1024 threads for each bin, but we have to somehow come up with a good number for the number of blocks. I

reasoned that, since we can fit about 2 blocks on one multiprocessor (SM) on the Udacity AWS server, then the optimal number of blocks should be the number of SMs multiplied by 2¹.

The code for this final optimization looked like so:

```
// retrieve the thread's global id -> don't need to check since we have  
// less threads*blocks than elements  
const int gid = blockIdx.x * blockDim.x + threadIdx.x;  
  
// declare external shared memory  
extern __shared__ unsigned int s_histo[];  
  
// zero out histogram and sync  
s_histo[threadIdx.x] = 0;  
__syncthreads();  
  
// iterate over data in chunks for each thread and atomically add to local histogram  
// and sync  
for(int i = gid; i < n; i += blockDim.x * gridDim.x)  
    atomicAdd(&(s_histo[g_vals[i]]), 1);  
__syncthreads();  
  
// atomically add local histogram into global histogram  
atomicAdd(&(g_histo[threadIdx.x]), s_histo[threadIdx.x]);
```

When launched with 1024 threads and SM_COUNT * 2, its runtime averaged at 1.6 ms. Thus, this achieved a 5x speedup from the naïve code.

Although this implementation is pretty speedy, I think that it could be improved by lowering the number of threads per block (perhaps to 256?) in order for less threads to access the local histogram per block, and this will result in less idle time per thread. If we had 256 threads per block instead of 1024, then I believe this could be achieved if every thread updated $1024 / 256 = 4$ bins instead of only 1 bin, as we see above. I'm sure that the use of atomic operations is also slowing down the runtime, but this particular approach of computing a histogram needs to use atomics because race conditions would occur otherwise.

¹ Although I know that that the max number of threads per SM on the Udacity server is only 1536 (and each block in our code has 1024), the runtime for SM_COUNT blocks was about the same as SM_COUNT * 2 blocks, and so I kept it that way. If we lower the threads per block to 256 as suggested in the final paragraph, we could fit about 6 blocks on one SM and would then try a block size of SM_COUNT * 6.